MoodWave

See your music as a cloud.

MoodWave is a small desktop music app written in Python. Pick a song from your library and watch a soft pastel cloud-sphere breathe, swell and shimmer to the bass, the high notes, the beat and the tempo, with a simple duration bar along the bottom.

What you get

- A **title page** with a **Play** button and a **Library** button
- A **library** that lists your songs; click one to play it
- A **cloud-sphere visualiser** in soft baby blue, lilac, pink and mint
- A small **duration bar** showing how far through the song you are
- **Drag and drop** to add songs, plus pause, fullscreen and keyboard shortcuts
- Each song is analysed once and remembered, so replays start instantly

MoodWave does not include any music. Use your own files (mp3, wav, flac or ogg).

Quick start

You need **Python 3.10 or newer**. If you don't have it, download it from python.org/downloads.

1. Get the code

Easiest: on the GitHub page click the green **Code** button, then **Download ZIP**, and unzip it somewhere you can find it (for example your Desktop).

Or with git:

```
None
git clone https://github.com/YOUR-USERNAME/moodwave.git
```

(Replace `YOUR-USERNAME` with the account the project lives on.)

2. Set it up and run it

Pick your system. You only do the setup once.

1. **macOS** Press **Cmd + Space**, type **Terminal**, press Enter.
2. Go into the project folder (change the path if you put it somewhere else):

```
None
cd ~/Desktop/moodwave
```

3. Create a private space for the project's libraries (a "virtual environment"):

```
None
python3 -m venv venv
```

4. Switch it on. Your prompt should now start with `(venv)`:

```
None
source venv/bin/activate
```

5. Install the libraries (this takes a minute or two):

```
None
pip install -r requirements.txt
```

6. Run MoodWave:

```
None
python3 moodwave.py
```

Next time, you only need steps 2, 4 and 6.

1. **Windows** Install Python from python.org/downloads. On the first screen of the installer, **tick "Add python.exe to PATH"**.
2. Open the project folder in File Explorer, click the address bar at the top, type `cmd` and press Enter. A Command Prompt opens already inside the folder.

3. Create a private space for the project's libraries (a "virtual environment"):

```
None
python -m venv venv
```

4. Switch it on. Your prompt should now start with `(venv)`:

```
None
venv\Scripts\activate
```

5. Install the libraries (this takes a minute or two):

```
None
pip install -r requirements.txt
```

6. Run MoodWave:

```
None
python moodwave.py
```

   (If `python` isn't recognised, try `py moodwave.py` instead.)

Next time, open Command Prompt in the folder again and repeat steps 4 and 6.

> Tip: use **Command Prompt** rather than PowerShell. PowerShell can block the `activate` step with a permissions message.

Linux Same as macOS (`python3 -m venv venv`, `source venv/bin/activate`, `pip install -r requirements.txt`, `python3 moodwave.py`). You may also need your distribution's SDL and libsndfile packages if pip reports a missing library.

Adding songs

There are three ways, and they all end up in the same place:

1. **Drag and drop** audio files (or a whole folder of them) onto the MoodWave window. They're copied into the `music` folder.
2. Click the Open **music folder** in the Library and drop the files in yourself.
3. Put audio files **next to `moodwave.py`**. MoodWave finds those automatically too.

Supported formats: **mp3, wav, flac, ogg**. (m4a/AAC files aren't supported; convert them to mp3 or wav first.)

Controls

Where	**Key / mouse**	**What it does**
Title page	**Enter** or click **Play**	Play the selected song
Title page	**L** or click **Library**	Open the library
Title page	**Esc**	Quit
Library	**Up / Down**, **Enter**	Move through the list and play
Library	Click a song, mouse wheel	Play it, scroll the list
Library	**Esc** or **Back**	Return to the title page
Playing	**Space** (or click the cloud)	Pause / resume
Playing	**Esc** or **Back**	Stop and go back
Anywhere	**F**	Toggle fullscreen

How it works

When you pick a song, MoodWave first **listens to the whole file once** and writes down, 60 times a second, what the music is doing. Then, while the song plays, the cloud simply looks up those numbers for the current moment.

What MoodWave hears	**What the cloud does**
Bass (low notes)	The whole cloud swells, and the bottom puffs grow the most

What MoodWave hears	What the cloud does
Treble (high notes)	The top of the cloud shimmers, rotation speeds up a little, and the colours drift
26 pitch bands (low to high)	Each puff listens to one band by height: bottom puffs follow bass, top puffs follow treble
Beats	The whole cloud breathes in with a soft pulse, then eases back out
Loudness	The surface rolls faster and billows more
Tempo (BPM)	Sets how fast the sphere turns

The cloud swells **quickly** but settles **slowly**, which is what gives it a cloud-like feel instead of a jittery one.

Analysis results are saved in a hidden `.moodwave_cache` folder. Delete that folder any time to force songs to be re-analysed.

Make it your own

Everything below is near the top of `moodwave.py` or in the cloud code. Each section of the file starts with a plain-English note, so it reads like a guided tour.

I want to change...	Look for...
The colours	`PALETTE_STOPS` and the colour constants at the top
The size of the cloud	`min(w, h) * 0.175` in the `cx, cy, R = ...` line of `draw_play`
How many puffs	`CloudSphere(n=420)`; lower it if your computer struggles
How strongly it reacts	the numbers in `r_eff = ...` inside `CloudSphere.draw`
How slowly it settles	`down = 1 - math.exp(-3.0 * dt)` in `CloudSphere.update` (smaller = slower)

I want to change...	Look for...
The starting window size	`self.windowed_size = (1100, 750)` in `App.__init__`

Troubleshooting

`python3: command not found` (Mac) or `'python' is not recognized` (Windows) Python isn't installed or isn't on your PATH. Install it from python.org/downloads (on Windows, tick "Add python.exe to PATH"), then open a new Terminal / Command Prompt. On Windows you can also try `py` instead of `python`.

`can't open file '...moodwave.py'` Your Terminal isn't inside the project folder. Use `cd` to go into it, then check with `ls` (Mac) or `dir` (Windows) that you can see `moodwave.py`.

`ModuleNotFoundError: No module named 'pygame'` (or `librosa`, `numpy`) The virtual environment isn't switched on, or the libraries weren't installed. Run the "switch it on" step so your prompt starts with `(venv)`, then `pip install -r requirements.txt`.

pygame fails to build with an error like `'SDL.h' file not found` This happens on very new Python versions, where the original `pygame` has no ready-made download. This project uses **pygame-ce**, which does. Make sure you installed from `requirements.txt`, and if the old one sneaked in: `pip uninstall pygame` then `pip install pygame-ce`.

An mp3 won't load Most systems can read mp3 out of the box. If yours can't, install ffmpeg: on Mac run `brew install ffmpeg` (see Homebrew); on Windows run `winget install ffmpeg` or download it from ffmpeg.org. A `.wav` file is a quick way to check whether the file or the setup is the problem.

"That song didn't load" on the loading screen The file may be damaged or in a format MoodWave can't read. The technical reason is printed in the Terminal window.

My downloaded file is called `moodwave (1).py` Your browser added `(1)` because a copy already existed. Rename it to `moodwave.py` (or delete the old one first).

The first song takes a few seconds That's the one-time analysis. Songs you've played before open instantly.

Project layout

```
None
moodwave/
├── moodwave.py          the whole app (heavily commented)
├── requirements.txt     the libraries to install
├── README.md            this file
├── .gitignore           keeps your music and cache out of git
├── screenshots/         images used in this README
├── music/               your songs (created on first run, not committed)
└── .moodwave_cache/     saved analysis (created automatically, not committed)
```

Built with

- pygame-ce for the window, drawing, keyboard/mouse and music playback
- librosa for listening to the audio (frequencies, beats, tempo)
- NumPy for the maths behind the cloud
- python-soundfile for reading audio files

Helpful resources

These are the links I found useful while building each part. The same links appear in the matching section of `moodwave.py`.

GitHub projects

- librosa: audio analysis library (docs: librosa.org/doc)
- pygame-ce: the community edition of pygame (docs: pyga.me/docs)
- NumPy and python-soundfile
- Similar projects worth a look: ASCIIMusicVisualizer (mel spectrogram to visuals), animusic (different frequencies drive different effects), baclaudio (pygame + librosa player that analyses a song before playing), AudioVisualizer (particle visualiser reacting to bass, mids and highs), Whirling (numpy + pygame visualisers), OpenLightShow (music-reactive effects with beat detection)

Audio analysis

- librosa.stft: splitting sound into frequencies
- librosa.feature.melspectrogram: grouping frequencies into bands
- librosa.onset.onset_strength: spotting beats and hits
- librosa.beat.beat_track: estimating tempo
- The mel scale: why pitch bands are spaced the way they are

The cloud (maths and graphics)

- Evenly distributing points on a sphere and a short Fibonacci lattice explainer: how the puffs are spread over the sphere
- 3D rotation: spinning the sphere
- 3D projection and perspective projection step by step: flattening 3D onto the screen
- Exponential smoothing: the quick-to-swell, slow-to-settle motion
- Smoothstep: the easing curve used to blend colours
- NumPy broadcasting (an older docs version, but the idea is unchanged)

The app

- The game loop pattern: what `run()` and `frame()` are doing
- pygame.mixer.music and pygame events (including drag-and-drop)
- Python threading and pathlib

Setting up Python

- Download Python, plus Python on Windows and macOS
- Virtual environments (venv)
- Homebrew docs (Mac package manager) and ffmpeg downloads